Kafka en acción: identificación de componentes

Sistema React + dos aplicaciones Spring Boot. Tabla basada en el código real de los repositorios y sus cambios.

Rutas: P = str-producer/src/main/java/com/lta/backend/; C = str-consumer/src/main/java/com/lta/backend/.
Cada celda de archivo se resuelve concatenando ese prefijo con la ruta indicada. Paquete base: com.lta.backend.

Componente	Archivo y clase	Función / configuración	Papel y lectura del código
Producer	P/services/StringProducerService.java **StringProducerService** C/listeners/StrConsumerListener.java **StrConsumerListener**	sendMessage(message) listener2(record) KafkaTemplate.send(...)	Publica solicitudes en str-topic: si contiene "menu", P1; si no, P0. listener2 publica el menú en str-topic-response. En la entrega el envío se delega a KafkaRequestReplyService y se conserva UUID como clave.
Consumer	C/listeners/StrConsumerListener.java **StrConsumerListener** P/services/ResponseListener.java **ResponseListener**	listener1(): group-1, P0 listener2(): group-1, P1 listener3(): group-2, todo str-topic listen(record): respuestas	listener1 solo registra. listener2 llama OrderProcessor.process() y responde. listener3 observa independientemente. ResponseListener original usaba group-1; ahora usa product-replies-${random.uuid} para completar cada petición propia.
Topics	P/config/KafkaAdminConfig.java **KafkaAdminConfig**	kafkaAdmin() topics() con KafkaAdmin.NewTopics	Originales: str-topic y str-topic-response. Nuevos: product-query-request y product-query-response. Se crean al iniciar el gateway. test-topic es solo un ejemplo manual; __consumer_offsets es un topic interno.
Particiones	P/config/KafkaAdminConfig.java P/services/StringProducerService.java C/listeners/StrConsumerListener.java	topics(): .partitions(2) sendMessage(): elige P0/P1 @TopicPartition(partitions={0/1})	Cada uno de los cuatro topics tiene P0 y P1. Los listeners originales 1 y 2 tienen asignación fija. El nuevo grupo se suscribe al topic completo. El orden se garantiza dentro de una partición, no entre las dos.
Réplicas	P/config/KafkaAdminConfig.java **KafkaAdminConfig.topics()** docker-compose.yml, servicio kafka	.replicas(1) en los cuatro topics KAFKA_OFFSETS_TOPIC_ REPLICATION_FACTOR: 1	Factor 1 = una copia por partición, incluyendo al líder. Hay un broker y no existe redundancia ante su caída. La variable del contenedor configura el topic interno de offsets; no reemplaza replicas(1) de los topics de aplicación.
Listeners	C/listeners/StrConsumerListener.java C/config/StringConsumerConfig.java P/services/ResponseListener.java	@KafkaListener y @TopicPartition consumerFactory() validMessageContainerFactory() validMessage(); listen(record)	Los listeners son consumidores administrados por Spring. validMessage() es un interceptor: aunque el log dice "ignorado", devuelve siempre el record y NO filtra. KAFKA_LISTENERS del broker configura sockets de red, no métodos de aplicación.

Hallazgo del original: ResponseListener.response era una variable estática compartida. Podía mezclar respuestas de consultas simultáneas. La ampliación usa una clave UUID y un future por solicitud. La clase clientes.java no participa en los flujos observados.

La ampliación: consulta de productos

La respuesta vuelve por Kafka y completa la misma petición HTTP. El navegador no se conecta al broker y no se usa WebSocket.

Archivo / clase nueva o extendida	Función relevante	Comportamiento implementado
P/resources/ProductQueryController.java **ProductQueryController**	query(query)	GET /api/products?query=...; máximo 80 caracteres. Delega a Kafka, retorna CompletableFuture<String> con JSON.
P/services/KafkaRequestReplyService.java **KafkaRequestReplyService**	request(topic, partition, message) complete(requestId, value)	UUID como clave Kafka; mapa de futures por petición. Hasta 128 consultas pendientes, espera 8 segundos, limpieza al terminar y fallo de publicación propagado.
C/listeners/ProductQueryListener.java **ProductQueryListener**	onQuery(record) process(record)	Nuevo @KafkaListener para product-query-request, grupo product-query-workers. Consulta catálogo y publica product-query-response con la misma clave; confirma envío antes de finalizar.
C/services/ProductCatalogService.java **ProductCatalogService**	search(query) normalize(text)	6 productos de demostración en memoria, precios MXN. Busca por ID, nombre o categoría; ignora acentos y mayúsculas; búsqueda vacía devuelve todos.
P/services/ResponseListener.java P/resources/ApiExceptionHandler.java	listen(record) invalid() / timeout() / unavailable()	Consume ambos topics de respuesta y completa el future por UUID. HTTP 400 ante consulta inválida; 504 si no llega respuesta; 503 ante error de envío.
Front: src/pages/mainpages/Home.jsx src/components/ProductQuery.jsx src/components/ArchitectureView.jsx	Home: setTab() / onTabKey() ProductQuery.search() ArchitectureView: setZoom()	Pestañas por teclado, tabla de resultados con estados, traza real UUID/partición/offset. Arquitectura integra SVG ampliable y permite descargar SVG/PNG.

Flujo que demuestra la implementación

ProductQuery.search() → HTTP GET /api/products → ProductQueryController.query() → KafkaRequestReplyService.request() → **product-query-request** → ProductQueryListener.onQuery() → ProductCatalogService.search() → **product-query-response** → ResponseListener.listen() → complete(UUID) → HTTP 200 → tabla React.

Verificación y límites

9 pruebas de back-end aprobadas, incluyendo HTTP + Kafka embebido, consultas concurrentes, búsqueda vacía/sin coincidencias, validación y timeout 504. **6 pruebas de navegador aprobadas** con el broker Docker: consulta real, pestañas/imagen, móvil y menú original. Solo el caso de error visual inyecta un HTTP 503. Particiones y réplicas verificadas con kafka-topics --describe.

Preparación original: el front no compilaba por react-router-dom no declarado; se agregó. Los dos back-ends originales sí respondieron el menú vegan. El sistema final se comprobó completo. Datos en memoria, RF1 y entrega con posibles reintentos: es una demostración local, sin base de datos ni garantía de procesamiento exactamente una vez.

Repositorios y fuentes consultadas

davidvaldezm/frontProducerKafka-modulo8 · rama actividad-modulo8

davidvaldezm/kafkaExample-modulo8 · rama actividad-modulo8

Apache Kafka: diseño, particiones y replicación · Spring Kafka: publicación y request/reply. Código original: front 7384e0c; back f57f607. El diagrama y las capturas acompañan este PDF.